Building a Data Pipeline to Support Analyzing Clickstream Data with AWS

Overview and objectives

Throughout various AWS Academy courses, you have completed hands-on labs. You have used different AWS services and features to build a variety of solutions.

In this project, you're challenged to use familiar AWS services, as well as AWS services that might be new to you, to build a solution. In many parts of the project, step-by-step guidance is *not* provided. This is intentional. These specific sections of the project are meant to challenge you to practice skills that you have acquired throughout your learning experiences prior to this project. In some cases, you might be challenged to use resources to independently learn new skills.

By the end of this project, you should be able to do the following:

- Deploy a data analytics pipeline on AWS that supports the analysis of website clickstream data.
- Transform clickstream data before it arrives in the visualization layer.
- Use AWS services to analyze clickstream data.
- Design a dashboard reporting mechanism for clickstream data analysis.
- Adjust the data analytics pipeline.

The lab environment and monitoring your budget

This environment is long lived. When the session timer runs to 0:00, the session will end, but any data and resources that you created in the AWS account will be retained. However, note that the public IP address of your café webserver will change. If you later launch a new session (for example, the next day), you will find that your work is still in the lab environment, but you will need to access EC2, and use the new public IP address to access and browse the café website. Also, at any point before the session timer reaches 0:00, you can choose **Start Lab** again to extend the lab session time.

Important: Monitor your lab budget in the lab interface. When you have an active lab session, the latest known remaining budget information displays at the top of this screen. The remaining budget that you see might not reflect your most recent account activity. If you exceed your lab budget, your lab account will be disabled, and all progress and resources will be lost. If you exceed the budget, please contact your educator for assistance.

AWS service restrictions

In this lab environment, access to AWS services and service actions are restricted to the ones that are needed to complete the lab instructions. You might encounter errors if you attempt to access other services or perform actions beyond the ones that are described in this lab.

Scenario

AnyCompany Café sells dessert and coffee items through their website. They have cafés in multiple cities throughout the world. The company wants to gain insights into the business by using data about how people interact with the website. The company plans to analyze clickstream data trends to make smarter decisions about where to invest. AnyCompany Café has hired your consulting company to lead this effort. You are a data integration specialist and work with a team of data engineers, data analysts, and web developers.

The clickstream log data from the café website includes an entry for every click that a potential customer makes while browsing the website. Your task is to design and create a data analytics pipeline to capture the clickstream data. You will also create an analytics dashboard so that the café owner can quickly observe customer behavior. The team wants to gain insights on user behavior on the website and then use those insights to focus advertising efforts. The company might even use the data to decide where to open additional locations.

Solution requirements

The solution must meet the following requirements:

- Design and cost-optimize the solution prior to building.
- Ensure that the solution is functional.
- Transform data.

- Ingest access_log data from the café web server.
- Use simulated log data.
- Analyze and visualize data.
- Generate insights.

Lab project tips

Knowledge of Linux Bash commands would be helpful but is not mandatory. Bash commands used in this lab include `cp`, `mv`, `cat`, `sudo`, `yum`, `wget`, and `find`. You are encouraged to familiarize yourself with the purpose of each command if you don't already know them.

Tip: For information about Linux Bash commands, see the Linux Man Pages on linux.die.net. To learn about a specific command, enter it in the search box at the top of the page.

Finally, you are encouraged to be resourceful as you complete this project. For example, reference the AWS Documentation or a search engine if you need an answer to a technical question.

As you work through the project you will find other tips to help you complete the project, specific to each phase or task.

Approach

The following table describes the phases of the project:

NUMBERED PHASE	DETAIL
1	Create an architecture diagram and cost estimate for the solution.
2	Use AWS Cloud9 to access the web server. Observe the access logs that are generated when you browse the website.
3	Install and configure the Amazon CloudWatch agent and the configuration file (httpd.conf) for the Apache web server so that access and error logs can be collected and sent to CloudWatch.

NUMBERED PHASE	DETAIL
4	Confirm that the web server generates logs that the CloudWatch agent collects and sends to CloudWatch.
5	Use simulated logs and ensure that CloudWatch receives these logs.
6	Use CloudWatch Logs Insights to query the access log group and generate visualizations.
7	Adjust the analytics pipeline to deliver new insights with visitor geolocation data. Create a dashboard that the café owner can use to make decisions.

Phase 1: Planning the design and estimating cost

In this phase, you will plan the design of your architecture. First, you will create an architecture diagram.

Next, you will estimate the cost of the proposed solution and present the estimate to your educator. An important first step for any solution is to plan the design and estimate the cost. Review the various components in the architecture to adjust the estimated cost. Cost, along with features and limitations of specific AWS services are important factors when building a solution. For example, these can help to determine solution components and an architecture pattern to use.

Task 1: Creating an architectural diagram

1. Create an architectural diagram to illustrate what you plan to build. Consider how you will accomplish each requirement in the solution. Read through the phases in this document to be aware of which AWS services and features that the company requested you to use. Be sure to include the following service or resource icons in your diagram:
    - CloudWatch log group
    - CloudWatch Logs Insights
    - CloudWatch dashboard
    - AWS Cloud9 environment (Amazon EC2) instance for the web server
    - AWS Identity and Access Management (IAM) role
    - Amazon S3

    References

    - AWS Architecture Icons: This site provides tools to draw AWS architecture diagrams.

- AWS Reference Architecture Diagrams: This site provides a list of AWS architecture diagrams for various use cases. You might want to use these diagrams as references.

2. Optional: Consider including additional AWS resources as part of your architecture diagram. For example:
    - An Amazon Simple Storage Service (Amazon S3) bucket to store a copy of the logs

Task 2: Developing a cost estimate

Develop a cost estimate that shows the cost to run the solution in the us-east-1 Region for 12 months. Use the AWS Pricing Calculator for this estimate.

Optional: Adding the diagram and cost estimate to a presentation

- Add your architectural diagram and cost estimate to presentation slides. Your educator might want to evaluate this information as part of assessing your work on this project. A presentation template is provided.
- In addition, you should capture screenshots of your work at the end of each task or phase to include in the presentation or document. Your instructor might use the presentation or document to help assess how well you completed the project requirements.

Reference

- PowerPoint presentation template

You have created an architecture diagram and cost estimate for your solution.

Phase 2: Analyzing the website and confirming weblog data

The café website already exists in your AWS account. In this phase, you will analyze the existing account resources. You will then verify that the café website is accessible. Finally, you will observe how clickstream data is generated when you access the website.

Task 1: Analyzing and understanding the lab environment

The first time that you start the lab environment, resources are created for you in the AWS account. As you read about each resource, you are encouraged to locate them and observe their settings by using the AWS Management Console.

The resources include the following:

- A virtual private cloud (VPC) named *Lab VPC* with a public subnet named *Public Subnet* in it. The subnet can reach the internet through an internet gateway.
- An AWS Cloud9 instance that runs as an EC2 instance in the public subnet. AWS Cloud9 provides an integrated development environment (IDE), which is convenient to edit files and run AWS Command Line Interface (AWS CLI) commands.
- A security group that is associated with the EC2 instance.
- An IAM role named *CafeRole*, which is attached to the EC2 instance. The role allows access from the instance to other AWS services, such as AWS Systems Manager, Amazon S3, and CloudWatch.
- The AWS Cloud9 instance also hosts the café web application. The web application runs on an Apache httpd web server and PHP, and it uses a MariaDB database that is also running on the instance.

❗ Important:

- When your lab session ends (because the timer runs to 0:00 or you choose **End Lab**), the AWS Cloud9 EC2 instance will be stopped. Resources that you have created in the account will remain, but you won't be able to access the web application while the instance is stopped.
- When you restart the lab (for example, the next day), the AWS Cloud9 EC2 instance will be stopped and it will need to be restarted. Refer to Stop and start your instance for details.
- You can reset the lab environment to the original configuration at any time by choosing **Reset** above these instructions.

  ⚠ Warning: Resetting the environment will *delete everything that you have created in the account*.

Task 2: Modifying a security group and verifying that the web application loads

The AWS Cloud9 instance that was created for you runs on an EC2 instance. That instance also hosts the café web application, which runs on TCP port 80. However, the security group that is attached to the instance doesn't yet allow inbound traffic on that port. In this task, you will resolve that issue.

1. In the Amazon EC2 console, locate the instance and adjust the security group so that it allows inbound TCP traffic on port 80 from any IPv4 location to the café website.
2. Copy the public IPv4 address of the EC2 instance.

   Tip: If the instance is not running, start it now so that a public IP will be assigned.

   Important: When your lab session restarts, the public IP address of the AWS Cloud9 instance changes. Be sure that you are using the latest public IPv4 address of the instance to access the café website.
3. Paste the IP address into a new browser tab and add `/cafe` to the end of the URL before loading it.

   Important: Ensure that you are loading *http://public-ip/cafe*, not *https://public-ip/cafe*.

   The café website loads and displays pictures of the desserts that the café offers. Keep this page open to use in a later task.

Task 3: Observing and backing up the httpd access_log

Now that you know that the web application is accessible, you will monitor the log that records every time a user accesses a page on the website.

1. In the AWS Cloud9 console, open the IDE for the AWS Cloud9 environment that was created for you.

   Note: The AWS Cloud9 IDE runs on the same EC2 instance that hosts the café web application. For more information about the AWS Cloud9 IDE, see Getting Started: Basic Tutorials for AWS Cloud9.
2. In the AWS Cloud9 terminal, use the Linux `tail` command to track changes in real time to the Apache web server's *access_log* file.

   Tips:

- Like many other applications that can be installed on Linux, the Apache httpd web server stores logs by default in a subdirectory of */var/log*.
- With the `tail` command, you will want to use the `-f` parameter. For more information about the command, see this Linux manual page for the tail command.

3. Return to the browser tab where the café web application is open, and refresh the page.

   Tip: Arrange the café website and the AWS Cloud9 IDE tab next to each other so that you can observe how log entries are made as you browse the café website.

4. Confirm that each time that you load a page on the café website, a new log entry is created in the access_log. Try a variety of actions—refresh the homepage, go to the **Menu** page, submit an order, or view the order history.

   - As you navigate the website, observe how the URL of the page changes. For example:
     - The main webpage is at *http://public-ip/cafe*.
     - The menu webpage is at *http://public-ip/cafe/menu.php*.
     - After you place an order, you are taken to a page at *http://public-ip/cafe/processOrder.php*.
   - The path of the page that you load is recorded in each log entry, along with other details. You are generating and observing clickstream data in real time.

5. To stop the `tail` command, press Ctrl+C. You are returned to the command prompt.

6. To back up the access_log file, run the following command:

```
sudo cp /var/log/httpd/access_log /home/ec2-
user/environment/initial_access_log
```

You now have a better understanding of how the café website works and how clickstream log entries are generated.

Phase 3: Installing the CloudWatch agent and creating the configuration file

In this phase, you will install and configure a CloudWatch agent on the café web server so that you can use CloudWatch logs to capture the access_log data as well as any error_log data.

Task 1: Installing the CloudWatch agent on the web server

To install the CloudWatch agent, run the following command in the AWS Cloud9 terminal:

```
sudo yum install -y amazon-cloudwatch-agent
```

Task 2: Creating the configuration file for the CloudWatch agent

1. To download and install a CloudWatch agent configuration file, run the following command:

```
wget https://aws-tc-largeobjects.s3.us-west-2.amazonaws.com/CUR-TF-200-ACCAP4-1-91575/capstone-4-clickstream/s3/config.json
sudo mv config.json /opt/aws/amazon-cloudwatch-agent/bin/
```

2. To review the contents of the config.json file, run the following command:

```
sudo cat /opt/aws/amazon-cloudwatch-agent/bin/config.json
```

Notice that this file contains configuration information for the agent, including the following:

- Which logs to collect from the café web server
- Which log group in CloudWatch to place the logs in
- How long to retain the logs
- Which metrics to collect and how to aggregate them

Note: This file was created by running `sudo /opt/aws/amazon-cloudwatch-agent/bin/amazon-cloudwatch-agent-config-wizard` and answering a series of questions to configure the CloudWatch agent. However, for this project, the configuration file is provided for you.

Task 3: Configuring the httpd.conf log format as JSON

Now that the CloudWatch agent is installed and configured, you will modify the default format of the access logs and error logs that the web server produces. Specifically, you will need to change the format from the default Common Log Format (CLF) to JSON format. This will allow the CloudWatch agent to send the log files to CloudWatch log groups.

1. Locate the web server's configuration file, which is named *httpd.conf*, and back up the file.

   Tip: The /etc directory and its subdirectories are a common place for applications to store configuration files. If you are not familiar with how to work with the Linux terminal and using commands, you may find it helpful to look up the details of the `cp`, `find`, and `sudo` commands in a Linux man pages reference.

2. Modify the *httpd.conf* file so that the error logs and access logs will be formatted as JSON instead of the default CLF format:

   - To avoid the need to use a terminal text editor (such as vi or nano) to edit the files in the */etc/httpd/conf* directory, run the following commands:

     ```
     ln -s /etc/httpd/conf /home/ec2-
     user/environment/httpdconf
     ```

     Note: The previous command creates a symlink to the */etc/httpd/conf* directory so that you can see the files in the AWS Cloud9 **Environment** window.

     ```
     sudo chown -R ec2-user /etc/httpd/conf
     ```

     Note: The previous command changes the ownership of the files in the */etc/httpd/conf* directory so that you can save changes when you use the AWS Cloud9 text editor. The ec2-user Linux user does not have permissions to edit the files in this directory.

   - In the **Environment** window of the IDE, expand the **httpdconf** folder, and open the **httpd.conf** file in the file editor.

3. Adjust the error log configuration:

   - In the *httpd.conf* file, go to the line that reads `ErrorLog "logs/error_log"` (around line 182). Comment out the line.

     Tip: To comment out a line, enter a `#` character at the beginning of the line.

   - Copy and paste the following text as a new line immediately after the line that you just commented out:

     ```
     ErrorLog "/var/log/www/error/error_log"
     ```

- Copy and paste the following text as a new line immediately after the `ErrorLog` line that you just added:

```
ErrorLogFormat "{\"time\":\"%{%usec_frac}t\",
\"function\" : \"[%-m:%l]\", \"process\" : \"
[pid%P]\" ,\"message\" : \"%M\"}"
```

4. Adjust the access log configuration:

- Go to the line that reads `<IfModule log_config_module>` (around line 191). Comment out ***the line below it***, which contains `LogFormat "%h %l %u %t \"%r\" %>s %b \"%{Referer}i\" \"%{User-Agent}i\"" combined`.
- Copy and paste the following text as a new line immediately after the `LogFormat "%h %l %u %t \"%r\" %>s %b" common` line that remains:

```
LogFormat "{ \"time\":\"%{%Y-%m-%d}tT%{%T}t.%
{msec_frac}tZ\", \"process\":\"%D\",
\"filename\":\"%f\", \"remoteIP\":\"%a\",
\"host\":\"%V\", \"request\":\"%U\",
\"query\":\"%q\",\"method\":\"%m\",
\"status\":\"%>s\", \"userAgent\":\"%{User-
agent}i\",\"referer\":\"%{Referer}i\"}" cloudwatch
```

- Comment out the entire `<IfModule logio_module>` section. To do this, comment out the three lines that weren't previously commented out.
- Go to the line that reads `CustomLog "logs/access_log" combined` (around line 219).
- Keep the line, but copy and paste the following text as a new line immediately after the `CustomLog` line.

```
CustomLog "/var/log/www/access/access_log"
cloudwatch
```

5. Save and close the file.

Task 4: Using the updated configuration file for the CloudWatch agent

1. Create new access and error log directories so that the directory locations that you specified in the httpd.conf file exist on the server.

Note: The web server can create the files, but you must define the directories first.

2. Restart the httpd (web server) service so that the changes that you made to the web server configuration are loaded.

   Tip: Use the `systemctl` command combined with the `sudo` command.

3. Start the CloudWatch agent so that it uses the *config.json* file that you installed.

   Tip: For information about running the `amazon-cloudwatch-agent-ctl` command, see Start the CloudWatch Agent Using the Command Line.

4. To confirm that the agent is now running and active, run the following command:

   ```
   service amazon-cloudwatch-agent status
   ```

You have now installed and configured the CloudWatch agent and httpd.conf files. You also created access and error log folders.

Phase 4: Testing the CloudWatch agent

In this phase, you will test the CloudWatch agent to confirm that it is collecting and sending the web server access log and error log data to CloudWatch.

Task 1: Observing the httdp access_log file

1. In the café web application, perform a variety of actions, as you did before to generate logs.
2. Use the same technique that you used in phase 2 task 3 to observe the actions as they are added to the access_log file. Remember that you configured the web server to write to a different log location, and log entries should now be written in JSON format instead of the default Common Log Format (CLF).

Task 2: Observing the amazon-cloudwatch-agent.log file

1. Determine the location of the *amazon-cloudwatch-agent.log* file, and use the `cat` command to see the contents of the file.
2. In the file, look for lines that are similar to the following:

```
[inputs.logfile] Reading from offset 10620 in
/var/log/www/error/error_log
[inputs.logfile] Reading from offset 19850 in
/var/log/www/access/access_log
[logagent] piping log from apache/error/i-
0d378cc2f1bb61140(/var/log/www/error/error_log) to
cloudwatchlogs with retention 180
[logagent] piping log from apache/access/i-
0d378cc2f1bb61140(/var/log/www/access/access_log) to
cloudwatchlogs with retention 180
```

These lines indicate that the CloudWatch agent is reading from the two web server log files and piping (sending) those logs to the CloudWatch service.

Tip: You can safely ignore any `error getting disk usage ("/sys/kernel/debug/tracing"): permission denied` lines that appear in the log.

Task 3: Placing an order in the web application and observing the CloudWatch logs

1. Open the café web application at `http://<public-ip-address>/cafe`

   Note: Replace <*public-ip-address*> with the public IP address of the web server instance.

   Tip: To make the log entries a bit different, use a mobile device to browse the website.

2. Go to the **Menu** page, and submit an order.
3. Verify that your latest actions were captured in CloudWatch:
   - In the CloudWatch console, locate the **apache/access** entry in the **Log groups** area.
   - Open the log stream.
   - Expand the first entry to see the log details, which are in JSON format. Verify that your latest actions were captured.

   Note: Your clickstream data is now available in CloudWatch because the CloudWatch agent on the web server writes this data to CloudWatch logs.

You have successfully tested the new clickstream logging solution with CloudWatch, after confirming that logs are received in the log stream.

Phase 5: Using the simulated log and ensuring that CloudWatch receives the entries

In this phase of the project, you will replace the existing access_log with one that has simulated data. This simulated log has many more entries than you could generate by manually browsing the website yourself. This larger dataset will help to simulate real-world log activity for a website that is accessed frequently.

After you replace the existing file with this simulated data, you will restart the CloudWatch agent so that it collects the new access_log data and sends it to CloudWatch for further analysis.

Task 1: Analyzing the simulated log file

1. Review the simulated log file:
    - To see the first few lines of the simulated log data, run the following command:

      ```
      cat samplelogs/access_log.log | head
      ```

    - To pretty-print the first line of the log file so that it's easier to read, run the following command:

      ```
      cat samplelogs/access_log.log | head -1 | python -m
      json.tool
      ```

      Notice that the format of the file matches the format of the logs that your web server generated after you changed the LogFormat setting in the httpd.conf file. This format matching is intentional.

    - To count the number of lines in the file, run the following command:

      ```
      cat samplelogs/access_log.log| wc -l
      ```

      As you can see, this log has a lot of data in it.

Task 2: Using the new log file

1. To stop the CloudWatch agent service, use the Linux `systemctl` command.

   Tip: For more information, see Stopping and Restarting the CloudWatch Agent.

2. Put the new simulated access log in the location where the CloudWatch agent expects to find it.

   Important: The new simulated access log must be named `access_log` (*not* `access_log.log`) for the agent to use it.

3. Restart the CloudWatch agent service so that it will send the new log entries to CloudWatch.

Task 3: Confirming that the new logs appear in the CloudWatch log group

Now that the new log file is in place and the CloudWatch agent service has been restarted, you will verify that the logs appear in CloudWatch.

In the CloudWatch console, locate the **apache/access** log group. In the log stream, confirm that many more entries appear in the stream than you saw previously.

Tip: An indication that the log group now contains the new log data is if the stream has multiple entries with the same timestamp.

You have confirmed that the simulated log data is now appearing in the log stream.

Phase 6: Using CloudWatch Logs Insights for analysis

Now that you have confirmed that the CloudWatch agent is collecting and sending the simulated log data to CloudWatch, you will use CloudWatch Logs Insights to run queries on the access_log data.

Your specific objective in this phase is to use the clickstream data to determine the number of visitors who accessed the menu but didn't purchase anything.

Task 1: Determining the number of visitors who accessed the menu

1. In the navigation pane of the CloudWatch console, choose **Logs Insights**.

Tip: For this task and later tasks, you might want to refer to Analyzing Log Data with CloudWatch Logs Insights.

2. Get the count of users who accessed the menu:
    - Set the time period of the query to match the time period that the access_log has data for.
    - Run the following query:

```
fields @timestamp, remoteIP
| filter request = "/cafe/menu.php"
| count(remoteIP) as visitors by @timestamp
| sort @timestamp asc
```

      The results are summarized on the **Logs** tab with a message in the following format (where X and Y are number values):

```
Showing 1000 of X records matched
Y records...scanned...
```

      The key data to notice is the number of records that matched (X) out of the total number of records that were scanned (Y).

    - Record this information in a new text file in the AWS Cloud9 IDE. Name the file `phase6-results.txt` and save it to the `~/environment` directory. Alternatively, if your instructor has asked you to create a presentation or document to record your work, include the information there.
3. Save the query with the following settings:
    - **Query name:** Enter `menu-visitors`
    - **Folder:** Choose **Create new** and then enter `non-geo-results`

      Notice that the saved query appears in the **Queries** panel.

Task 2: Determining the number of visitors who made a purchase

1. Get the count of users who made a purchase:
    - Set the time period of the query to match the time period that the access_log has data for.
    - Run query from the previous task, but change the filter request to `/cafe/processOrder.php`

- Record the results in your *phase6-results.txt* file, presentation, or document.

2. Save the query to the *non-geo-results* folder. Use `purchasers` for the query name.

Task 3: Determining the number of visitors who accessed the menu but didn't make a purchase

1. Use the results of the prior two tasks to determine how many people accessed the menu page but didn't make a purchase.

   Tip: The formula to get this count is as follows:
   Number of visitors who accessed the menu but didn't make a purchase = (Number of visitors who accessed the menu) - (Number of visitors who made a purchase)

2. Calculate the percentage of people who visited the menu page and also made a purchase.
3. Record these results in your *phase6-results.txt* file, presentation, or document.

You have successfully used queries in CloudWatch Logs Insights to determine how many website visitors completed certain actions.

Phase 7: Adjusting the pipeline to deliver new insights

In this final phase of the project, you will enhance the data in your pipeline to deliver new insights to the café owner.

Task 1: Understanding the requirements

You meet with the owner of the café and share the insights that you have gathered from the clickstream data. The owner is impressed with the solution that you have built so far, but they want to know more.

The café owner asks if you can modify the solution to provide the following:

- Geolocation information for website visitors.
- A visual dashboard to gain insights. The dashboard should include the following information:

    - A *pie chart* that shows the 10 cities that had the most website visitors who accessed the menu page.
    - A *log table* that shows the 10 cities that had the most website visitors who made a purchase.
    - A *pie chart* that shows the 10 regions that had the most website visitors who accessed the main page of the website.
    - A *bar chart* that shows the 10 regions that had the most website visitors who made a purchase.
- The ability to store the access logs in Amazon S3 and the ability to query the logs by using SQL

The geolocation information and visual dashboard will help the owner to analyze business trends and determine where to open new café locations. Being able to store the data in Amazon S3 and use SQL to query the data will provide flexibility to use a variety of tools and services to analyze the data in the future.-

You have new solution requirements. The remaining tasks in this phase provide additional detail to help you fulfill these requirements.

Task 2: Using the example logs that include geolocation information

In this task, you will replace the existing access_log with a file that includes geolocation information.

1. To analyze the sample access_log that includes geolocation information, run the following commands. The results will provide information about the data structure and the number of lines in the file:

    ```
    cd ~/environment
    head -1 samplelogs/access_log_geo.log | python -m json.tool
    cat samplelogs/access_log_geo.log | wc -l
    ```

2. Replace the existing access_log on the web server with the new log that contains geolocation information. Ensure that the CloudWatch agent will capture the data and send it to CloudWatch.

    Recall that you performed this task in phase 5, tasks 2 and 3. The differences this time are that you are going to have geolocation information in the new access_log, and you need to delete the existing CloudWatch log stream before you restart the CloudWatch agent.

At a high level, the steps are as follows:

- Stop the CloudWatch agent.
- Copy the new log to the location where the CloudWatch agent expects to find it. Ensure that the new log is named `access_log`.
- To confirm that the access_log file was replaced, use the following command:

```
sudo head -1 /var/log/www/access/access_log
```

- Verify that you see geolocation data ("lat" and "lon") in the log entries. The following is an example entry:

```
{ "time":"2023-02-22 03:38:18", "process":"5112",
"filename":"/var/www/html/cafe",
"remoteIP":"91.192.109.163", "host":"10.0.1.102",
"request":"/cafe", "query":"", "method":"GET",
"status":"200", "useragent":"Mozilla/5.0 (Windows NT
10.0) AppleWebKit/531.2 (KHTML, like Gecko)
Chrome/111.0.553.2.0.854.0", "referer":"-",
"country":"ES", "region":"Andalusia", "city":"La
Rinconada", "lat": "37.4867198", "lon":"-5.9814868"}
```

3. Delete the existing apache/access log group from CloudWatch logs, recreate the apache/access log group and restart the CloudWatch agent service.

Task 3: Building a dashboard to observe the geolocation data in CloudWatch Logs Insights

Reference

- Visualizing Log Data in Graphs

In this task, you will build a CloudWatch dashboard to meet the café owner's requirements.

1. **Create a CloudWatch dashboard** that is named `cafe-dashboard`.

   Tip: The Using Amazon CloudWatch Dashboards page might be helpful.

2. **Add a pie chart widget to the dashboard** to show which cities have the most visitors to the café website's menu page. Use the following settings:

- Widget type: **Pie**
- Data source: **Logs**
- Time period to query: **Custom** (Date that you started the project to today)
- Log groups to query: **apache/access**
- Query:

```
fields remoteIP, city
| filter request = "/cafe/menu.php"
| stats count() as menupopular by city
| sort menupopular desc
| limit 10
```

Analysis: The query will return log entries with the remoteIP and city fields from each log, when the user loaded the menu page, a provided through the filter of the request field for /cafe/menu.php. The query counts the number of visits to the menu page for each city and then sorts those cities in descending (desc) order. Therefore, cities with the most visits will show at the top of the results. Finally, the query limits the returned results to the top 10.

- Run the query. The pie chart appears.
- Choose **Create widget**. The widget is added to the dashboard.
- Hover on the **Log group: apache/access** title of the widget and choose the edit icon.
- Rename the widget as `Cities visiting the menu the most`
- Choose **Apply**.
- On the dashboard page, choose **Save** in the top-right corner.

3. Use the same approach to **add a second widget**. The widget should show the 10 cities that had the most website visitors who made a purchase:
    - Use a logs table.
    - Name the widget as `Cities placing the most orders`.
    - The widget should cover the same time period as the first widget.
    - The query should filter requests to the `/cafe/processOrder.php` page.

4. Use the same approach to **add a third widget**. The widget should show the 10 regions that had the most website visitors who accessed the main page of the website:
    - Use a pie chart.

- Name the widget as `Regions visiting the website the most`.
- The widget should cover the same time period as the first widget.
- The query should filter requests to the `/cafe` page.

5. Use the same approach to **add a fourth widget**. The widget should show the 10 regions that had the most website visitors who made a purchase:
    - Use a bar chart.
    - Name the widget as `Regions placing the most orders`.
    - The widget should cover the same time period as the first widget.
6. After you add all four widgets to the dashboard, make sure that you save the dashboard.

Note: If your instructor has asked you to create a presentation or document to record your accomplishments, save your queries and a screen capture of the final CloudWatch dashboard to your presentation or document.

Task 4: Saving the log file to an S3 bucket

In this task, you begin to fulfill the final requirement, as described in phase 7, task 1.

In the AWS Cloud9 terminal, use the AWS CLI to copy the geolocation access log from your `/var/log/www/access` directory or `~/environment/samplelogs` directory to the S3 bucket that was provided for you in the lab environment.

Note: The bucket name contains *logbucket*.

Tip: The AWS CLI Command Line Reference for Amazon S3 might be helpful to refer to.

Ending your session

Reminder: This is a long-lived lab environment. Data is retained until you either use the allocated budget or the course end date is reached (whichever occurs first).

To preserve your budget when you are finished for the day, or when you are finished actively working on the assignment for the time being, do the following:

1. At the top of this page, choose **End Lab**, and then choose **Yes** to confirm that you want to end the lab.

A message panel indicates that the lab is terminating.

Note: Choosing **End Lab** in this environment will *not* delete the resource you have created. They will still be there the next time you choose Start lab (for example, on another day).

2. To close the panel, choose **Close** in the upper-right corner.